

SE446: Big Data Engineering

Week 9: Spark MLlib — Exercises & Practice Questions

Spring 2026 — Prof. Anis Koubaa

Instructions: Answer all questions. For code questions, write complete, runnable PySpark code. Show your reasoning for conceptual questions. This exercise sheet covers all topics from Week 9.

1 Conceptual Questions — Traditional ML vs Distributed ML

- Q1. The Three Walls.** Explain the three limitations (“walls”) that prevent scikit-learn from handling very large datasets. For each wall, explain how Spark MLlib solves it.
- Q2. Same Math, Different Execution.** A student says: “Spark MLlib uses completely different algorithms than scikit-learn.” Is this statement correct? Explain what actually changes when you move from scikit-learn to Spark MLlib, and what stays the same.
- Q3. When NOT to Use Spark.** Give **three** concrete scenarios where using scikit-learn on a laptop would be a better choice than Spark MLlib on a cluster. Justify each.
- Q4. Distributed Training.** Consider training a Logistic Regression model with gradient descent on a 3-worker Spark cluster. Describe step by step what happens during **one iteration** of training:
- (a) What does each worker compute?
 - (b) What does the driver do?
 - (c) How are the results combined?

2 Pipeline API — Transformers & Estimators

- Q5. Transformer vs Estimator.** Classify each of the following as a **Transformer** or an **Estimator**. For Estimators, state what they produce when `.fit()` is called.

Component	Type	Output of <code>.fit()</code> (if Estimator)
VectorAssembler	_____	_____
StringIndexer	_____	_____
RandomForestClassifier	_____	_____
LogisticRegression	_____	_____
StandardScaler	_____	_____

- Q6. Pipeline Order.** A student writes the following pipeline:

```
pipeline = Pipeline(stages=[
    assembler,          # VectorAssembler
    crime_indexer,      # StringIndexer
    rf                  # RandomForestClassifier
])
```

- (a) Will this code run successfully? Why or why not?
- (b) What is the correct order of stages? Explain why.

Q7. Data Leakage. A student fits the `StringIndexer` on the **full dataset** before splitting into train and test sets:

```
fitted_indexer = crime_indexer.fit(df)          # full dataset!
df = fitted_indexer.transform(df)
train_df, test_df = df.randomSplit([0.8, 0.2])
```

- (a) What is the problem with this approach? Name the specific ML issue.
- (b) Rewrite this code correctly using a `Pipeline`.

Q8. Why Pipelines Matter More in Distributed ML. In scikit-learn, many practitioners skip pipelines and call `.fit()` and `.transform()` manually. Give **two reasons** why this shortcut is more dangerous in Spark MLlib than in scikit-learn.

3 Feature Engineering

Q9. StringIndexer Mapping. Given the following crime type frequencies in the training data:

Crime Type	Count
THEFT	3200
BATTERY	2800
NARCOTICS	1500
ROBBERY	900

What index will `StringIndexer` assign to each crime type? Explain the rule.

Q10. StringIndexer vs OneHotEncoder.

- (a) Explain the difference between `StringIndexer` and `OneHotEncoder`.
- (b) For a **Random Forest** model, is `OneHotEncoder` necessary after `StringIndexer`? Why or why not?
- (c) For a **Logistic Regression** model, why would using only `StringIndexer` (without `OneHotEncoder`) lead to poor results?

Q11. VectorAssembler. Given a `DataFrame` row with the following values:

District	crime_index	Hour	domestic_index
11	3.0	22	1.0

- (a) What will the `features` column contain after `VectorAssembler`?

- (b) Why does Spark MLlib require all features in a **single vector column**, unlike scikit-learn which accepts a matrix?

Q12. Feature Selection. We used four features: `District`, `PrimaryType`, `Hour`, `Domestic`.

- (a) Why did we **not** use the `Block` column (street address)?
- (b) Propose one additional feature you could derive from existing columns that might improve predictions. Explain your reasoning.

4 Model Training & Evaluation

Q13. Confusion Matrix Calculations. A model produces the following confusion matrix on the test set:

	Predicted: No Arrest	Predicted: Arrest
Actual: No Arrest	5400	200
Actual: Arrest	600	1800

Calculate the following metrics. **Show your work.**

- (a) Accuracy
- (b) Precision (for the Arrest class)
- (c) Recall (for the Arrest class)
- (d) F1 Score (for the Arrest class)

Q14. Why Not Just Accuracy? A dataset has 8,500 crimes with no arrest and 1,500 crimes with arrest.

- (a) What accuracy would a model achieve if it **always** predicts “No Arrest”?
- (b) What would the AUC-ROC of this model be?
- (c) Why is this model useless despite high accuracy?
- (d) Which metric would you use instead, and why?

Q15. Model Output Interpretation. The model produces the following output for a crime record:

```
prediction = 1.0
probability = [0.35, 0.65]
```

- (a) What does `[0.35, 0.65]` mean?
- (b) If you lower the decision threshold from 0.5 to 0.3, would the prediction change for this record? For a record with `probability = [0.62, 0.38]`?
- (c) How does lowering the threshold affect precision and recall in general?

Q16. Feature Importances. A Random Forest produces these feature importances:

Feature	Importance
<code>crime_index</code>	0.92
<code>domestic_index</code>	0.05
<code>District</code>	0.02
<code>Hour</code>	0.01

- (a) What does it mean that `crime_index` has importance 0.92?
- (b) Given this result, could you build a simpler model using **only** `crime_index` and get similar performance? Why or why not?
- (c) A colleague suggests removing `Hour` since its importance is only 0.01. What is the risk of this approach?

5 Model Comparison & Selection

Q17. Three-Model Comparison. You trained three models and obtained these results:

Metric	Logistic Reg.	Random Forest	GBT
AUC-ROC	0.65	0.86	0.86
Accuracy	0.66	0.82	0.81
F1 Score	0.59	0.81	0.81
Training Time	2s	3s	5s

- (a) Why does Logistic Regression perform so much worse than the tree-based models?
- (b) RF and GBT have nearly identical metrics. Which would you choose for production? Justify.
- (c) The accuracy of LR (0.66) is very close to the no-arrest rate (66%). What does this tell you about what LR actually learned?

Q18. Bagging vs Boosting.

- (a) Random Forest uses **bagging** and GBT uses **boosting**. Explain the fundamental difference between these two ensemble strategies.
- (b) Which approach parallelizes better on a Spark cluster? Why?
- (c) A student says: “GBT is always better than RF because it corrects errors.” Give a counter-example where RF would be preferred.

6 Code Exercises

Q19. Complete the Pipeline. Fill in the blanks to build a complete ML pipeline:

```
from pyspark.ml.feature import _____, _____
from pyspark.ml.classification import _____
from pyspark.ml import _____

# Step 1: Encode "Primary Type" -> numeric index
crime_indexer = _____(
    inputCol="_____",
    outputCol="crime_index",
    handleInvalid="skip"
)

# Step 2: Combine features into one vector
assembler = _____(
    inputCols=["District", "crime_index", "Hour"],
    outputCol="_____"
)

# Step 3: Define classifier
rf = RandomForestClassifier(
```

```

    featuresCol="_____",
    labelCol="_____",
    numTrees=100
)

# Step 4: Chain into pipeline
pipeline = _____(stages=[_____, _____, _____])

# Step 5: Train
model = pipeline._____(train_df)

# Step 6: Predict
predictions = model._____(test_df)

```

Q20. Evaluate the Model. Write PySpark code to:

- Compute the AUC-ROC score using `BinaryClassificationEvaluator`
- Compute accuracy and F1 using `MulticlassClassificationEvaluator`
- Print a confusion matrix using `groupBy`

Q21. CrossValidator. Write PySpark code to tune a Random Forest with the following grid:

- numTrees: 50, 100, 200
- maxDepth: 3, 5, 10

Use 3-fold cross-validation with AUC-ROC as the evaluation metric.

Hint

Use `ParamGridBuilder`, `CrossValidator`, and `BinaryClassificationEvaluator`. Set `parallelism=4` for parallel model training.

Q22. Debug This Code. The following code has **three errors**. Find and fix each one.

```

from pyspark.ml import Pipeline
from pyspark.ml.feature import StringIndexer, VectorAssembler
from pyspark.ml.classification import RandomForestClassifier

# Error 1 is here
assembler = VectorAssembler(
    inputCols=["District", "Primary Type", "Hour"],
    outputCol="features"
)

# Error 2 is here
rf = RandomForestClassifier(
    featuresCol="features",
    labelCol="Arrest"
)

# Error 3 is here
pipeline = Pipeline(stages=[assembler, rf])
model = pipeline.fit(df)
predictions = model.transform(df)

```

For each error, explain:

- What the error is
- What exception or incorrect result it causes
- How to fix it

7 Hyperparameter Tuning & Production

Q23. CrossValidator Math. You configure a `CrossValidator` with:

- Parameter grid: `numTrees = [50, 100]`, `maxDepth = [3, 5, 10]`
- `numFolds = 5`
- `parallelism = 3`

- (a) How many total models will be trained?
- (b) How many “rounds” of parallel training will occur?
- (c) If each model takes 30 seconds to train, estimate the total wall-clock time.

Q24. Save and Load.

- (a) Write PySpark code to save a trained `PipelineModel` to HDFS.
- (b) Write code to load it back and make predictions on new data.
- (c) Explain why saving to HDFS is better than saving to local disk in a cluster environment.
- (d) The saved model includes fitted `StringIndexer` mappings. Why is this important? What breaks if you save only the classifier?

Q25. Overfitting.

- (a) You set `maxDepth=20` for a Random Forest and observe training accuracy = 0.99 but test accuracy = 0.78. What is happening?
- (b) Name **two** hyperparameters you could adjust to reduce overfitting in a Random Forest. For each, explain what value change helps and why.
- (c) How does `regParam` in Logistic Regression serve a similar purpose?

8 Bonus — Ethics & Real-World Application

Q26. Algorithmic Bias. Your arrest prediction model learns that District 11 has a 60% arrest rate while District 1 has only 15%.

- (a) If police use this model to allocate patrol resources, what feedback loop could emerge?
- (b) Is the model learning about **crime patterns** or about **policing patterns**? Explain the difference.
- (c) Propose one concrete mitigation strategy.

Q27. Real-Time vs Batch. Suppose a police department wants to predict arrests in **real-time** as crimes are reported (latency < 100ms).

- (a) Would you use Spark MLlib for real-time inference? Why or why not?
- (b) Describe an alternative architecture: how would you train and deploy the model?

End of Exercises — Total: 24 Questions

SE446 — Big Data Engineering — Spring 2026 — Prof. Anis Koubaa